



PRD_08_集成测试与调试...
文件



PRD_09_多设备治理与扩...
文件



README.md
文件



PRD_01_产品总览与模块...
文件



PRD_02_打印控制与任务...
文件



PRD_03_设备接入与设备...
文件



PRD_04_文件管理与切片...
文件



PRD_05_软件设置与参数...
文件



PRD_06_设备诊断与维护...
文件



PRD_07_告警与历史记录...
文件

我先给你上传部分现在打印软件的prd文档供你分析当前项目进展，你可以先阅读这些文档数据，因为后面我还会上传设计文档部分



DEV_07_设备诊断与维护...
文件



DEV_08_错误码告警历史...
文件



DEV_09_集成测试台能力...
文件



DEV_10_多设备通信治理...
文件



README.md
文件



DEV_00_产品与设计文档...
文件



DEV_01_项目总架构与模...
文件



DEV_02_配置与参数分层...
文件



DEV_03_打印控制闭环与...
文件



DEV_04_设备接入与多设...
文件



DEV_05_文件管理与切片...
文件



DEV_06_软件设置与全局...
文件

这些是软件的设计文档



UI_04_软件设置与参数页...
文件



UI_05_设备诊断与维护页...
文件



UI_06_集成测试台详细设...
文件



README.md
文件



UI_01_主界面与集成测试...
文件



UI_02_打印监控页设计说...
文件



UI_03_文件管理与切片交...
文件

这些事UI文档



关于创建PRD之前问题答...
文件

1. 对于最小可行性技术闭环中，要保证模型切片后有下表面支撑，普通模型的下表面支撑要在最小闭环中体现。
2. 针对 全彩UV体素切片软件技术选型方案_初版.md 中13节的确定事项，我总结了一个文档，你可以分析上传文件更新技术选型方案给我。供我审核，判断初版的PRD、技术方案的书写以及demo如何实现。



SlicerDemo(1).zip

压缩归档

这是一个单材料的切片软件demo，请你分析这个软件的技术栈是什么。针对我们的切片软件是否可以吸取相关技术，



REPORT_00_P0_Demo...

文件

这个是Codex生成的当前现状，请你根据这个文档进行后续文档的生成



1. 当前项目完成00A阶段，引用的文档信息为Codex生成的当前项目状态信息
2. 现阶段进行浮雕模型的切片，切片数据出现问题。在Codex中进行咨询，引用的图片是Codex的回复。
3. 针对浮雕模型问题，要采用什么方式进行解决？解决该问题是否在后续计划中？如果在后续计划中，是否可以延后处理该问题？
4. 我们是应该针对浮雕模型进行专项处理？还是继续按照我们后续计划推进项目处理？



REPORT_00_P0_Demo...
文件

在实施完00C阶段，进行测试发现，模型切片数据中只有光油材料，并没有支撑材料在其中显示？在之前的00B 00A阶段中，每个切片数据都是由模型材料和支撑材料2中组成，Codex的回复原因如下

00C 浮雕配置是：

```
"slicingMode": "relief_heightfield",  
"modelMaterial": {  
  "materialChannel": "v"  
},  
"support": {  
  "enabled": false
```

展开 ∨



REPORT_00_P0_Demo...
文件



ROADMAP_后续PRD_D...
文件

已经完成00C阶段修复，可生成对于的支撑、



REPORT_00_P0_Demo...

文件



ROADMAP_后续PRD_D...

文件

已经完成00C阶段修复，可生成对于的支撑、光油材料。引用文件是Codex生成的当前项目状态和后续文档生成几乎，请判断当前阶段完成后，那后续的计划应该如何实现？请生成对应的计划和文档



REPORT_01_Relief当前...
文件

这是Codex生成的当前阶段的文档信息，请你根据这个文档信息生成后续的执行计划。



REPORT_02_支撑与孤岛...
文件

1. 已完成02阶段编码工作，是否继续03阶段工作？还是你根据引用文件以及项目代码进行进一步修改？
2. 项目当前现实状态如引用文档中所述。
3. 详细代码已上传代码仓库，你可以进行查阅



REPORT_03_RGBWSV...

文件

1. 已完成03阶段编码工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为03阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



REPORT_04_彩色纹理切...
文件

1. 已完成04阶段编码工作，判断是否继续生成后续阶段文档信息？或根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为04阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



REPORT_04A_纹理阶段...
文件

1. 已完成04A阶段编码工作，判断是否继续生成后续阶段文档信息？或根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为04阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅

在RIP阶段的代码中，读取文件出现这个提示



REPORT_05_材料策略当...

文件

这是05阶段的状态报告，请你结合这份报告判断实现需要将之前生成的03B阶段文档进行修改？



PRD_03B_v0.2_RGBWS...

文件

1. 已完成03B阶段编码工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为03B阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



PRD_03B_v0.2_RGBWS...
文件

1. 已完成03B阶段编码工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为03B阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



REPORT_03C_回归与RI...

文件

1. 已完成03C阶段编码工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为03C阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



REPORT_06_3MF与OBJ...

文件

1. 已完成06阶段编码工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为06阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



REPORT_06A_3MF兼容...

文件

1. 已完成06A阶段编码工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修

改？

2. 引用文档信息为06A阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



REPORT_06A_3MF兼容...
文件

1. 已完成06A阶段编码工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为06A阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



REPORT_06B_3MF纹理...
文件

1. 已完成06B阶段编码工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为06B阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



REPORT_05A_真实材料...
文件

1. 已完成05A阶段编码工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为05A阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



f9dcf066-c355-4372-8d...
文件

1. 已完成07阶段编码工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？

2. 引用文档信息为07阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



REPORT_07A_Qt参数编...
文件

1. 已完成07A阶段编码工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为07A阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



REPORT_07B_UI自动化...
文件

1. 已完成07B阶段编码工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为07B阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



REPORT_07B_R1_UI真...
文件

1. 已完成07B-R1阶段编码工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为07B-R1阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅
4. 请你判断是否进入R0阶段，是否输出相关设计文档。是否将之前光油、彩色切片策略约束在R0中。关于这两个策略是否整理为相关文档进行说明，关于R1 R2的文档以及任务是否现在进行制定。



REPORT_R0_正式项目架...
文件

1. 已完成R0阶段工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为R0阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅



rules.md

文件



PRINTSOLUTION_AI_S...

文件



.agents.7z

文件

引用中的md文件是之前项目自己制定的SKILL，.agent压缩包是现在切片项目的SKILL。你能否仿照之前的SKILL，将现在切片项目的SKILL进行修改，从而满足或大部分满足之前的项目SKILL功能。



REPORT_R1_核心模块边...
文件

1. 已完成R1阶段工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为R1阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅，具体分支为 `r1-architecture-refactor`



REPORT_R2_配置报告测...
文件

1. 已完成R2阶段工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为R2阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅，具体分支为 `r1-architecture-refactor`



REPORT_08A_支撑桥接...
文件

1. 已完成08A阶段工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为08A阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅，具体分支为 `r1-architecture-refactor`



REPORT_09_OpenVDB...

文件

1. 已完成09阶段工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为09阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅，具体分支为 `spike/09-openvdb-sdf-kernel`



REPORT_09A_OpenVD...

文件

1. 已完成09A阶段工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为09A阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅，具体分支为 `spike/09-openvdb-sdf-kernel`



REPORT_09A_R1_Ope...

文件

1. 已完成09A-R1阶段工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为09A-R1阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅，具体分支为 `spike/09-openvdb-sdf-kernel`



REPORT_09A_R2_Ope...

文件

1. 已完成09A-R2阶段工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？

2. 引用文档信息为09A-R2阶段的报告信息。

3. 详细代码已上传代码仓库，你可以进行查阅，具体分支为 `spike/09-openvdb-sdf-kernel`



DEV_09B_OpenVDB_S...

文件

1. 已完成09B阶段工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？

2. 引用文档信息为09B阶段的报告信息。

3. 详细代码已上传代码仓库，你可以进行查阅，具体分支为 `spike/09B-openvdb-surface-shell-texture`



REPORT_09B_R1_真实...
文件

1. 已完成09B-R1阶段工作，判断是否继续生成后续阶段文档信息？还是你根据引用文件以及项目代码进行进一步修改？
2. 引用文档信息为09B-R1阶段的报告信息。
3. 详细代码已上传代码仓库，你可以进行查阅，具体分支为 `spike/09B-R1-real-model-shell-texture`

已思考 24s >

是的，新补充的正式切片软件总览已经囊括了后续一系列需求和开发计划，但它的定位是 **MASTER 总纲**，不是替代每个阶段的详细 PRD / DEV / TASKS。

换句话说：

MASTER PRD / DEV：负责定义整体方向、产品边界、模块分层、阶段路线、进入条件、红线
阶段 PRD / DEV / TASKS：负责指导某一个阶段具体实现



目前这版已经覆盖了什么

这两份文档已经覆盖了切片软件整体范围，而不是只覆盖 OpenVDB：

输入: STL / OBJ / MTL / PNG / 3MF / Texture2D / ColorGroup
几何: Legacy layer mask / OpenVDB SDF / shell / interior / support / varnish
材料: RGB / W / S / V / base / shell / support / varnish
输出: RGBWSV TIFF / manifest / RIP Reader / preview / report
配置: SliceConfig / Profile / Schema / Migration
UI: Qt Debug UI / 参数编辑 / overlay / report viewer
工程: CMake / scripts / unit / smoke / golden / CI / benchmark
后续: 09B-R2 / 09P / 09P-R1 / 09P-R2 / 09C / 09D / 10

文档包在这里:

SliceSoft_正式切片软件_MASTER_PRD_DEV文档包.zip

其中:

PRD_MASTER_SliceSoft_正式切片软件产品需求总览.md

DEV_MASTER_SliceSoft_正式切片软件总体架构与实现路线.md

后续开发计划是否已经包含

包含了。当前总路线被整理成:

09B-R2: 真实模型鲁棒性、性能/内存与多材质策略收口
→ 09P: OpenVDB production pipeline 接入设计
→ 09P-R1: experimental production implementation
→ 09P-R2: production hardening
→ 09C: SDF compensated varnish prototype
→ 09D: SDF support clearance / overhang diagnostics
→ 10: RIP / 设备 / 工艺联调阶段

其中 09B-R2 之前仍属于 **production** 接入资格验证, 09P 才开始做 正式生产切片链路接入设计。

这版够不够作为总 PRD / 总 DEV

作为总览和需求划分, 够用。

它已经能让 Codex 和开发者理解:

项目最终目标是什么
当前处于哪一步
哪些能力已经完成
哪些能力仍未完成
OpenVDB 在整体切片软件中处于什么位置
RGBWSV 协议哪些不可变
后续阶段如何拆分
production 接入前置条件是什么
哪些事情不能提前做

但它还不能替代后续每个阶段的详细文档。例如进入 09P 时，仍然需要单独生成：

```
PRD_09P_OpenVDB生产Pipeline接入设计.md
DEV_09P_OpenVDB与LegacyPipeline融合设计.md
TASKS_09P_OpenVDB生产接入任务清单.md
CODEX_PROMPT_09P_OpenVDB生产接入执行指令.md
```

这版还可以补强什么

如果你希望它成为真正的“项目总控文档”，我建议后续再补 3 个附属文件：

```
ROADMAP_MASTER_SliceSoft_阶段路线与验收门槛.md
MODULE_MATRIX_SliceSoft_模块能力状态矩阵.md
REQUIREMENT_TRACE_SliceSoft_需求到阶段映射表.md
```

其中最有价值的是 `REQUIREMENT_TRACE`，它可以把每个需求映射到：

```
需求编号
所属模块
当前状态
实现阶段
对应 PRD
对应 DEV
对应测试
是否 production 必需
是否已完成
```

当前判断

你现在这两份 MASTER 文档已经可以作为后续开发总纲使用。后续阶段文档应该全部引用它们：

```
docs/slicer/PRD_MASTER_SliceSoft_正式切片软件产品需求总览.md
docs/slicer/DEV_MASTER_SliceSoft_正式切片软件总体架构与实现路线.md
```

结论：它已经囊括了后续需求和开发计划，但属于总纲级；进入每个后续阶段时，仍需要继续生成阶段级 **PRD / DEV / TASKS**。

当前项目已全部提交到github，那后续要如何进行下一步操作？项目后续的几个步骤分别是什么？当前项目处于什么阶段？后续要如何一步步调整？

已查找可用工具 ∨

已调用工具



已思考 1m 34s >

是的，现在项目已经进入一个新的操作阶段：不再只是“继续写某个 demo”，而是要把 GitHub 上的阶段成果整理成稳定开发主线，并继续推进 09B-R2。

部分更早上上传的历史附件已经过期；下面判断基于当前 GitHub 仓库中可读取的 09B-R1 报告与我们刚补充的 SliceSoft 总 PRD / DEV 方向。

当前项目处于什么阶段

当前项目处于：

09B-R2：真实模型鲁棒性、性能/内存与多材质策略收口



不是 09P，也不是 production 接入阶段。

原因是 09B-R1 已经完成了真实 OBJ/3MF 壳层纹理链路：OBJ/MTL/PNG 或 3MF Texture2D → SceneModel → TriangleMeshData + UV/material → OpenVDB level set → shell/interior → BVH nearest triangle → barycentric UV → texture/diffuse/fallback RGB → report v2 / preview，但报告明确说明该链路仍是实验原型，没有接入 production slicer_cli 或 RGBWSV TIFF。

09B-R1 的自动验证已经通过：OBJ/3MF/fallback/negative cases、generated-box 回归、OpenVDB smoke、USE_OPENVDB=OFF Debug build、CI quick 均为 PASS。但它仍然只验证了闭合 OBJ/3MF fixture，不等价于复杂生产模型全面通过；真实指甲浮雕、薄壁、小间隙、尖角、自交、重复面和超大网格还未验证。

所以当前定位是：

OpenVDB / SDF 壳层纹理真实链路已打通；
现在正在验证它是否具备进入 production pipeline 设计的资格。



GitHub 提交完成后的下一步

你现在应该先做 仓库状态整理，再继续推进 09B-R2。

第一步：确认主线与实验分支边界

建议不要把所有 spike 分支直接合并进 main。当前最好保持：

main：
稳定主线，legacy slicer / RGBWSV / RIP / Qt Debug UI 基础能力保持可用



spike/09B-R1-real-model-shell-texture:

真实 OBJ/3MF 壳层纹理验证完成点

spike/09B-R2-shell-robustness-performance:

当前继续开发分支

如果 09B-R2 分支还没建，建议从 09B-R1 切出：

</> Bash



```
git checkout spike/09B-R1-real-model-shell-texture
git pull
git checkout -b spike/09B-R2-shell-robustness-performance
git push -u origin spike/09B-R2-shell-robustness-performance
```

如果 09B-R2 已经存在，就直接：

</> Bash



```
git checkout spike/09B-R2-shell-robustness-performance
git pull
```

第二步：把 MASTER 文档作为总控文档提交

你刚补的正式切片软件总文档应该放入：

docs/slicer/



建议文件为：

```
PRD_MASTER_SliceSoft_正式切片软件产品需求总览.md
DEV_MASTER_SliceSoft_正式切片软件总体架构与实现路线.md
```



它们不是替代阶段文档，而是后续所有阶段的总控文档。

后续 Codex 每次做阶段任务时，应该固定先读：

```
docs/slicer/PRD_MASTER_SliceSoft_正式切片软件产品需求总览.md
docs/slicer/DEV_MASTER_SliceSoft_正式切片软件总体架构与实现路线.md
```



再读当前阶段文档，比如：

```
docs/slicer/TASKS_09B_R2_鲁棒性能与多材质策略任务清单.md
docs/slicer/CODEX_PROMPT_09B_R2_鲁棒性能与多材质策略执行指令.md
```



第三步：继续完成 09B-R2

09B-R2 是当前最重要的一步。它不是新增 production 输出，而是补足真实复杂模型验证。

建议拆成 8 个小任务。

09B-R2-1：真实业务 Golden Fixtures

准备真实业务模型：

```
真实指甲 OBJ + MTL + PNG
真实指甲 3MF + Texture2D
复杂浮雕 OBJ/3MF
多材质/多纹理模型
UV seam 模型
薄壁/尖角/小间隙模型
```



目标是从“盒体 fixture”升级到“真实业务 fixture”。

09B-R2-2：拓扑鲁棒性诊断

在当前 `boundaryEdges / nonManifoldEdges / signedVolume` 基础上，增加：

```
duplicateFaces
oppositeDuplicateFaces
localWindingInconsistency
selfIntersectionPairs
connectedComponents
zeroVolumeComponents
minEdgeLengthMm
minTriangleAreaMm2
maxTriangleAspectRatio
thinFeatureWarnings
```



09B-R2-3：Scale-aware tolerance

不要继续只依赖固定 epsilon。需要根据：

```
模型 bbox
voxelSizeMm
绝对下限
相对尺度上限
```



计算：

```
position_epsilon_mm
area_epsilon_mm2
```



```
tie_epsilon_mm  
self_intersection_epsilon_mm
```

09B-R2-4: UV seam / Material seam / Tie-break

当前真实纹理转移已经能跑，但 seam 场景还要稳定化。

建议规则：

最近三角形命中：

1. 最小 distance
2. 最大 barycentric interior margin
3. 最小 source_triangle_index

UV seam：

命中哪一侧 triangle，就使用哪一侧 UV，不跨 seam 平均

Material seam：

命中 triangle 的 material 是唯一来源，不跨 material 混色

09B-R2-5: BVH 与 Texture Cache 统计

补充：

```
queryCount  
visitedNodes  
testedTriangles  
maxVisitedNodes  
bvhNodeCount  
bvhEstimatedBytes  
textureCacheHits  
textureCacheMisses  
textureCacheBytes  
perMaterialSampledVoxels  
perTextureSampledVoxels
```

09B-R2-6: Release 性能与内存基线

Debug 数据只能用于诊断，不能作为生产性能结论。09B-R2 要新增 Release benchmark：

```
1k triangles  
10k triangles  
50k triangles  
100k triangles (环境允许时)
```

记录：


```
importMs
adapterMs
topologyMs
levelSetMs
bvhBuildMs
transferMs
previewMs
totalMs
OpenVDB grid memory
BVH memory
texture cache memory
mask memory
process peak working set
```



09B-R2-7: Voxel / Shell Thickness Matrix

至少验证：

```
voxelSizeMm = 0.10 / 0.05 / 0.025
shellThicknessMm = 0.05 / 0.10 / 0.20
```



检查：

```
shell 单调性
inside 体积趋势
转移距离
内存增长
运行时间增长
小特征保留
```



09B-R2-8: 生成状态报告

完成后必须生成：

```
docs/slicer/REPORT_09B_R2_壳层纹理鲁棒性能与多材质策略当前状态.md
```



报告必须判断：

```
是否进入 09P production pipeline 接入设计
是否还需要 09B-R3
是否应先做 09C / 09D
```



第四步：09B-R2 完成后的分叉判断

09B-R2 完成后有两种路线。

路线 A：09B-R2 通过

进入：

09P: OpenVDB production pipeline 接入设计



09P 只做设计，不直接写 production 代码。它要解决：

surface_shell_texture 如何进入 SliceConfig
OpenVDB geometry kernel 如何进入 SlicePipeline
MaterialPolicy 如何接收 shell/interior 结果
shell RGB / interior base 如何映射 RGBWSV
失败时 fail_fast 还是 fallback_legacy
report schema 如何升级
golden package 如何比较
Qt Debug UI 如何展示结果



路线 B：09B-R2 不通过

不要进入 09P。进入：

09B-R3：针对性修复阶段



例如：

09B-R3-topology：拓扑鲁棒性修复
09B-R3-performance：性能/内存专项
09B-R3-seam：UV/material seam 专项



第五步：09P 之后才进入 production experimental

09P 设计通过后，再做：

09P-R1: experimental production path



这一阶段才允许在显式配置下接入 `slicer_cli`：

```
geometry_kernel.enabled = true  
texture.apply_mode = surface_shell
```



但必须保持：

默认 legacy path 不变
USE_OPENVDB=OFF 默认仍可构建



p0.rgbwsv.2 不变
R G B W S V 通道顺序不变
RIP Reader 可读
失败策略清晰

第六步：09P-R2 做 production hardening

09P-R1 跑通后，进入：

09P-R2: production hardening

这一步做：

配置 schema 固化
profile 固化
Qt Debug UI 展示
report/golden 固化
CI matrix 固化
真实模型集回归
RIP Reader 兼容回归
错误/fallback 策略收口

第七步：再展开 09C / 09D / 10

不要把这些提前混进 09B-R2。

后续顺序建议：

09C: SDF compensated varnish prototype
09D: SDF support clearance / overhang diagnostics
10: RIP / 设备 / 工艺联调

09C 依赖 shell/interior/distance。09D 依赖 SDF distance、surface normal、overhang angle。10 才是设备、RIP、喷头通道和工艺链路。

当前最推荐的实际操作顺序

你可以按这个顺序执行：

1. 确认 09B-R1 已提交并推送
2. 确认 MASTER PRD / DEV 已提交
3. 切换或创建 09B-R2 分支

4. 让 Codex 先读 MASTER PRD / DEV
5. 再让 Codex 执行 09B-R2 阶段任务
6. 完成真实模型 golden / topology / seam / benchmark / matrix
7. 生成 REPORT_09B_R2
8. 根据 REPORT_09B_R2 决定进入 09P 或 09B-R3

给 Codex 的提示可以这样写：

当前项目已完成 09B-R1，并已补充 SliceSoft MASTER PRD/DEV。
请在 spike/09B-R2-shell-robustness-performance 分支继续 09B-R2。

必须先阅读：

docs/slicer/PRD_MASTER_SliceSoft_正式切片软件产品需求总览.md
docs/slicer/DEV_MASTER_SliceSoft_正式切片软件总体架构与实现路线.md
docs/slicer/REPORT_09B_R1_真实OBJ_3MF壳层纹理验证当前状态.md
docs/slicer/TASKS_09B_R2_鲁棒性性能与多材质策略任务清单.md

本阶段只做真实模型鲁棒性、性能/内存、多材质与 seam 策略收口。

不要接入 production slicer_cli。

不要写 production RGBWSV TIFF。

不要修改 p0.rgbwsv.2。

完成后生成：

docs/slicer/REPORT_09B_R2_壳层纹理鲁棒性性能与多材质策略当前状态.md

最终判断

当前项目状态可以概括为：

基础切片软件架构已成型；
RGBWSV 输出协议已冻结；
OBJ/3MF/纹理/材料/支撑/Qt/CI 已有阶段性基础；
OpenVDB 真实依赖和真实 OBJ/3MF 壳层纹理链路已跑通；
现在处于 production 接入前的真实复杂模型鲁棒性与性能验证阶段。

所以后续不要急着进 production。最稳的路线是：

09B-R2
→ 09P
→ 09P-R1
→ 09P-R2
→ 09C / 09D
→ 10

当前最该做的是：把 09B-R2 做扎实，让它成为“是否允许 OpenVDB 壳层纹理进入正式切 pipeline”的决策依据。

来源

刚才没有提交r2阶段的代码，现在已经同步到github中。

1. 请阅读相关文件，分析当前处于什么阶段，后续要进行什么操作？当前是否仍未demo版本，已经完成了什么需求和功能。
2. 判断后续的任务都有哪些，要分原子任务、阶段任务进行总结
3. 将总结的任务输出到文档中，供我本地Codex可以直接阅读，并继续R2后续操作

已查找可用工具 ▾

已调用工具

▾

正在调用工具

^

已思考 2m 31s >

我已读取 `spike/09B-R2-shell-robustness-performance` 分支的 R2 报告、CMake、鲁棒性诊断、容差、测试脚本和 benchmark 脚本。更早上传的部分历史附件已经过期；本次判断以 GitHub 当前分支内容为准。

当前阶段判断

当前处于：

09B-R2：真实模型鲁棒性、性能/内存与多材质策略收口



R2 报告明确写到：当前“实验链路已完成一轮实现与验证”，可以作为 OpenVDB 表面壳层纹理鲁棒性/性能基线，但尚不能进入生产 RGBWSV 输出或 09P 生产化。

所以当前不是直接进入 production 的阶段。下一步建议进入：

09B-R3：壳层纹理生产准入前的拓扑精确诊断、稳定错误码、纹理边界与内存峰值收口



R2 报告也明确建议“先进入 09B-R3，而不是直接进入 09P 生产化”，重点处理 narrow-phase 自相交、稳定错误码、repeat/wrap 纹理、Windows 进程峰值内存和真实模型拓扑准入策略。

当前是否仍是 demo 版本

准确说：整个切片软件已经不是单纯 demo，但 OpenVDB 表面壳层纹理仍是实验/demo/diagnostic path。

当前 R2 新增的 `surface_shell_robustness_demo`、`surface_shell_robustness_unit_tests`、`run_surface_shell_robustness_tests.ps1`、`run_surface_shell_benchmarks.ps1` 都是实验链路和验证链路；CMake 中 `USE_OPENVDB` 默认仍为 OFF，OpenVDB 只有显式打开时才查找和链接。

R2 报告也明确写到，本轮没有修改 production `slicer_cli`、RGBWSV schema、通道顺序、位深、极性、MaterialPolicy、SupportShapePipeline 或生产切片输出；R2 输出只服务于实验报告、preview 和 benchmark，不写入生产 TIFF 包。

已完成的需求和功能

R2 已补齐了很多关键验证项：

真实指甲 OBJ / 3MF golden 样例验证
scale-aware tolerance
拓扑鲁棒性诊断
seam 与 multi-material 策略
BVH / texture cache / memory / benchmark 指标
checklist、golden expected JSON、回归脚本和阶段报告



这些都是 R2 阶段目标中明确列出的能力。

真实模型方面，OBJ golden 与 3MF golden 都已跑通：OBJ 有 70,262 个三角形，3MF 有 75,596 个三角形，均生成 inside/shell/interior 统计，并以 `nonProduction=true` 记录。

多材质 seam 策略方面，当前规则已经明确：BVH 找最近三角，tie-break 顺序为 distance、barycentric interior margin、triangle index；命中后只使用该三角的 UV/material/texture，不跨 UV seam 平均，也不跨 material seam 混色。

性能方面，Release benchmark 已覆盖 `bench_1k / bench_10k / bench_50k`，并输出 level set、BVH build、transfer、估算内存、BVH 节点和 tested triangles 等数据；但当前还不是严格生产性能门槛。

仍未完成的关键项

R2 报告列出的未完成项非常明确：

narrow-phase self-intersection 未完成
repeat/wrap texture fixture 未完成



```
process peak working set 未完成
stable warning/error code 未完成
真实 OBJ/3MF 跨格式一致性未完成
真实模型 production strict_closed 准入未通过
```

这些问题会直接影响是否能进入 09P production 接入设计。

代码层面也能看到当前自相交仍主要是 AABB overlap candidate：

```
MeshRobustnessDiagnostics 统计 self_intersection_pairs 和
self_intersection_sampled，但报告中也说明它不是完整 narrow-phase 三角相交。
```

已生成给本地 Codex 使用的文档包

[下载 REPORT09B_R2后_R3后续操作与任务拆分文档包.zip](#)

建议解压到：

```
docs/slicer/
```

包含：

1. [DOC_09B_R2_当前状态判断与后续操作路线.md](#)
2. [TASKS_09B_R3_R2后续原子任务与阶段任务清单.md](#)
3. [CODEX_PROMPT_09B_R3_R2后续执行指令.md](#)

后续阶段任务拆分

建议不要继续在 09B-R2 中无边界补丁，而是切出：

```
spike/09B-R3-shell-production-readiness
```

R3 分为这些阶段任务：

```
09B-R3-0：分支与文档同步
09B-R3-1：实现 narrow-phase triangle-triangle self-intersection
09B-R3-2：引入稳定 ValidationErrorCode / WarningCode
09B-R3-3：建立 repeat/wrap 纹理 fixture
09B-R3-4：接入 Windows process peak working set
09B-R3-5：真实模型拓扑生产准入策略评估
09B-R3-6：Benchmark 扩展与阈值建议
09B-R3-7：全量回归
09B-R3-8：生成 REPORT_09B_R3
```

其中最关键的原子任务是：

TriangleIntersectionQuery.h/.cpp
ValidationIssue.h/.cpp
surface_shell_repeat_texture.json
ProcessMemoryStats
DOC_DECISION_09B_R3_真实模型拓扑生产准入策略.md
REPORT_09B_R3_壳层纹理生产准入前诊断策略收口当前状态.md



给 Codex 的下一步

本地 Codex 可以直接读取：

docs/slicer/CODEX_PROMPT_09B_R3_R2后续执行指令.md



核心要求是：

不要接入 production slicer_cli
不要写 production RGBWSV TIFF
不要修改 p0.rgbwsv.2
不要替换 legacy texture path
不要把 warn_and_attempt 结果宣称为 production safe



完成 R3 后，再根据 REPORT_09B_R3 判断：

是否进入 09P
是否还需要 09B-R4
是否可以并行 09C



当前最稳妥路线是：

09B-R2 已完成一轮基线
→ 09B-R3 解决生产准入前的精确诊断与策略问题
→ 通过后进入 09P production pipeline 接入设计



来源

如果按照 OpenVDB production 接入必须满足 的情况，那后续还需要经历哪些阶段进行处理？ |

1. 可按照R3阶段已完成的情况进行处理，列出09阶段的后续阶段，已经相关阶段的要完成什么任务，已经任务的具体下分是什么。

2. 可以参照09B-R1/2阶段的相关文档进行输出，完成输出后告知每个阶段要进行哪些工作

正在查找可用工具 ▾

已思考 40s >

可以。下面我按“**09B-R3 已完成并通过**”这个前提来规划后续阶段。实际执行时仍要先读取 `REPORT_09B_R3...` 做 Gate 审查；如果 R3 报告没有通过，就不能进入 09P，而应继续 09B-R4。

我也已把后续阶段路线、阶段任务和原子任务整理成文档包，供本地 Codex 直接阅读：

下载 09阶段_R3后OpenVDB生产接入后续阶段任务规划文档包.zip

包含：

1. ROADMAP_09_R3后OpenVDB生产接入阶段路线与任务拆分.md
2. TASKS_09_R3后后续阶段任务总表.md
3. CODEX_PROMPT_09_R3后OpenVDB生产接入后续阶段总指令.md

当前依据

R2 报告已经明确：09B-R2 可以作为 OpenVDB 表面壳层纹理鲁棒性和性能基线，但尚不能进入生产 RGBWSV 输出或 09P 生产化。

R2 已完成真实指甲 OBJ/3MF golden、scale-aware tolerance、拓扑鲁棒性诊断、seam/multi-material 策略、BVH/texture cache/memory/benchmark 指标等内容，但仍限定为实验 OpenVDB 路径，不改 production `slicer_cli` RGBWSV TIFF 链路。

R2 报告最后建议先进入 09B-R3，而不是直接进入 09P，原因包括 narrow-phase self-intersection、稳定错误码、repeat/wrap fixture、process peak working set 和真实模型拓扑生产准入策略仍需收口。

R3 完成后的后续阶段

按 R3 已完成的前提，后续路线建议是：

09B-R3-GATE

→ 09P

→ 09P-R1

→ 09P-R2



→ 09P-R3
→ 09P-R4

并行但隔离的后续几何原型：

09C: SDF compensated varnish prototype
09D: SDF support clearance / overhang diagnostics

Stage 0: 09B-R3-GATE，生产接入前置条件审查

这个阶段不是开发新功能，而是判断 R3 是否真的允许进入 09P。

要完成：

读取 REPORT_09B_R3
确认 narrow-phase self-intersection 已完成
确认 ValidationErrorCode / WarningCode 已稳定
确认 repeat/wrap texture fixture 已通过
确认 process peak working set 已接入
确认真实模型 topology production admission 策略明确
确认 OpenVDB ON / OFF 全量回归通过
确认 p0.rgbwsv.2、channel order、legacy slicer_cli 默认路径未被破坏

通过后进入 09P。未通过则进入：

09B-R4：针对未通过项继续修复

Stage 1: 09P，OpenVDB production pipeline 接入设计

这是 设计阶段，不直接写 production TIFF，也不应该改 slicer_cli 生产链路。

要完成：

DOC_DECISION_09P_OpenVDB生产Pipeline接入设计.md
PRD_09P_OpenVDB生产Pipeline接入.md
DEV_09P_OpenVDB与LegacyPipeline融合设计.md
DEMO_09P_OpenVDB生产接入验证方案.md
TASKS_09P_OpenVDB生产接入设计任务清单.md
CODEX_PROMPT_09P_OpenVDB生产接入设计执行指令.md

核心任务拆分：

```
09P-1: Production 接入边界
09P-2: Config Schema 设计
09P-3: Pipeline 架构设计
09P-4: RGBWSV 映射设计
09P-5: Report / Golden / CI 设计
09P-6: Qt UI 展示设计
```



这一阶段要回答：

```
OpenVDB geometry kernel 如何进入 SlicePipeline?
surface_shell_texture 如何进入 SliceConfig?
shell RGB / interior fill 如何映射到 RGBWSV?
失败时 fail_fast、diagnostic_only、fallback_legacy 如何处理?
report schema、golden、RIP Reader、Qt UI、CI 如何配套?
```



Stage 2: 09P-R1, experimental production path 最小接入

这个阶段开始写代码，但必须是 **显式 feature flag** 下的 experimental path。

默认 legacy path 不变。

核心任务：

```
09P-R1-1: Config 落地
09P-R1-2: GeometryKernelService
09P-R1-3: SurfaceShellTextureService
09P-R1-4: slicer_cli experimental path
09P-R1-5: 初步 RGBWSV 写入
09P-R1-6: Smoke / experimental golden
```



要实现：

```
geometry_kernel.enabled = true
texture.apply_mode = surface_shell
```



在这种配置下，`slicer_cli` 可以进入 OpenVDB experimental path，生成 RGBWSV package、manifest 和 report。

但必须保持：

```
默认 legacy path 不变
USE_OPENVDB=OFF 构建不受影响
p0.rgbwsv.2 不变
R G B W S V 通道顺序不变
```



OpenVDB 不可用时报明确错误
fallback 不得静默

Stage 3: 09P-R2, RGBWSV / RIP / Golden 输出收口

09P-R1 只是“能输出”。09P-R2 要把它变成“可比较、可回归、可解释”。

核心任务：

```
09P-R2-1: RGBWSV Golden
09P-R2-2: RIP Reader 兼容
09P-R2-3: MaterialChannelComposer 收口
09P-R2-4: Failure / Fallback 收口
09P-R2-5: CI Matrix
```



要建立：

```
OBJ surface_shell RGBWSV golden
3MF surface_shell RGBWSV golden
missing texture golden
no UV golden
topology failure golden
RIP Reader compatibility check
material composition report
failure/fallback report
OpenVDB ON / legacy OFF CI matrix
```



这一阶段结束后，OpenVDB experimental production path 应该具备可回归能力，但仍不建议默认开启。

Stage 4: 09P-R3, Qt UI / Profile / Report / CI 工程化

这一阶段做工程化与可用性收口。

核心任务：

```
09P-R3-1: Profile
09P-R3-2: Qt Debug UI
09P-R3-3: Report Viewer
09P-R3-4: CI 与文档
```



要完成：

surface shell experimental profile
voxel size / shell thickness / mesh policy / failure policy 配置展示
Qt UI 显示 OpenVDB availability
Qt UI 显示 shell/interior preview
Qt UI 显示 topology diagnostics
Qt UI 显示 transfer source counts
Qt UI 显示 warning/error codes
report summary viewer
OpenVDB environment setup 文档
troubleshooting 文档
UI smoke 更新

这一阶段的重点是让工程师和测试人员能理解输出、定位失败，而不是只靠命令行和JSON。

Stage 5: 09P-R4, Production Gate 与 Release Candidate

这是生产准入判断阶段。

核心任务：

09P-R4-1：真实模型集验收
09P-R4-2：性能与内存验收
09P-R4-3：Production 开关决策
09P-R4-4：发布候选报告

要判断：

真实指甲 OBJ/3MF 集合是否通过
复杂浮雕集合是否通过
多材质集合是否通过
错误拓扑集合是否有明确策略
10k / 50k / 100k benchmark 是否满足预期
process peak working set 是否可接受
哪些 profile 可以启用 surface_shell
默认是否仍保持 legacy
是否允许 fallback_legacy
是否可标记为 production experimental 或 production capability

产出：

REPORT_09P_R4_OpenVDB生产准入当前状态.md

这个报告才是判断是否可以继续进入设备/RIP/工艺联调的依据。

Stage 6: 09C, SDF compensated varnish prototype

09C 可以在 09P 后期并行，但必须保持 prototype 隔离。

目标：

基于 OpenVDB/SDF distance 与 shell/interior 能力，验证 compensated varnish 的几何可行性

任务：

定义光油目标：表面增强、补偿、边缘控制
定义与 RGB shell 的关系
定义与 W/S channel 的关系
从 level set 获取 surface distance
估算 surface normal
支持 offset distance
生成 varnish mask
输出 preview
输出 report
支持 generated fixture
支持真实 OBJ/3MF diagnostic

红线：

不直接写 production V channel
不修改 p0.rgbwsv.2
不与 09P-R1/R2 的 RGBWSV 接入混在一个阶段

Stage 7: 09D, SDF support clearance / overhang diagnostics

09D 也是 prototype / diagnostic 阶段，不替换现有 SupportShapePipeline。

目标：

利用 SDF distance、surface normal 和 geometry diagnostics，为后续支撑 clearance、overhang

任务：

定义 overhang 诊断目标
定义 clearance 诊断目标
定义与 SupportShapePipeline 的边界
定义 report schema
从 SDF 或 mesh 估算 normal
根据 build direction 计算 overhang angle
输出 overhang mask
计算 support candidate 与 model distance

输出 clearance violation
输出 preview/report

红线：

不修改 SupportShapePipeline 默认行为
不修改 S channel production 输出
不替换当前支撑生产路径

给 Codex 的后续执行方式

让 Codex 先读取：

docs/slicer/REPORT_09B_R3_壳层纹理生产准入前诊断策略收口当前状态.md
docs/slicer/ROADMAP_09_R3后OpenVDB生产接入阶段路线与任务拆分.md
docs/slicer/TASKS_09_R3后后续阶段任务总表.md
docs/slicer/PRD_MASTER_SliceSoft_正式切片软件产品需求总览.md
docs/slicer/DEV_MASTER_SliceSoft_正式切片软件总体架构与实现路线.md

然后先执行：

09B-R3-GATE

Gate 通过后，不要直接写代码，先生成并完成：

09P: OpenVDB production pipeline 接入设计

最终路线总结

如果 R3 已经完成并通过，那么后续 09 阶段应该这样走：

09B-R3-GATE：生产接入前置条件审查
→ 09P：生产接入设计
→ 09P-R1：experimental production path 最小接入
→ 09P-R2：RGBWSV / RIP / golden 输出收口
→ 09P-R3：Qt UI / Profile / Report / CI 工程化
→ 09P-R4：Production gate / release candidate

之后再根据情况进入：

09C: SDF compensated varnish prototype
09D: SDF support clearance / overhang diagnostics

核心原则是：**R3** 通过后可以进入 **production** 接入设计，但不能跳过 **09P** 直接写 **production RGBWSV**。 